

Artificial Intelligence

Simplify machine learning with XGBoost and Amazon SageMaker

by Eitan Sela, Saksham Saini, Yash Pant, and Yijie Zhuang | on 21 JUN 2018 | in [Amazon SageMaker](#), [Artificial Intelligence](#), [Top Posts](#) | [Permalink](#) | [Comments](#) | [Share](#)

January 2021: Post updated with changes required for SageMaker SDK v2, courtesy of Eitan Sela, Senior Startup Solutions Architect

Machine learning is a powerful tool that has recently enabled use cases that were never previously possible—computer vision, self-driving cars, natural language processing, and more. Machine learning is a promising technology, but it can be complex to implement in practice. In this blog post, we explain [XGBoost](#)—a machine learning library that is simple, powerful, and able to address a wide variety of use cases. We also provide a step-by-step tutorial for running XGBoost on [Amazon SageMaker](#) on a sample dataset, showing you how to build a model that's able to predict whether or not a person will default on their credit card payment.

What is XGBoost?

XGBoost (extreme gradient boosting) is a popular and efficient open-source implementation of the gradient-boosted trees algorithm. *Gradient boosting* is a machine learning algorithm that attempts to accurately predict target variables by combining the estimates of a set of simpler, weaker models. By applying gradient boosting to decision tree models in a highly scalable manner, XGBoost does remarkably well in machine learning competitions. It also robustly handles a variety of data types, relationships, and distributions. It provides a large number of hyperparameters—variables that can be tuned to improve model performance. This flexibility makes XGBoost a solid choice for various machine learning problems.

Problems and use cases addressed by XGBoost

The three problems XGBoost most commonly solves are classification, regression, and ranking:

- **Classification.** The goal in classification is to take input values and organize them into two or more categories. An example classification use case is fraud detection. In fraud detection, the goal is to take information about the transaction and use it to determine if the transaction is either fraudulent or not fraudulent. When XGBoost is given a dataset of past transactions and whether or not they were fraudulent, it can learn a function that maps input transaction data to the probability that transaction was fraudulent.
- **Regression.** In regression, instead of mapping inputs to a discrete number of classes, the goal is output a number. An example regression problem is predicting the price that a house will sell for. In this case, when XGBoost is given historical data about houses and selling prices, it can learn a function that predicts the selling price of a house given the corresponding metadata about the house.
- **Ranking.** Suppose you are given a query and a set of documents. In ranking, the goal is to find the relative importance of the documents and order them based on relevance. An example use case of ranking is a product search for an ecommerce website. You could leverage data about search results, clicks, and successful purchases, and then apply XGBoost for training. This produces a model that gives relevance scores for the searched products.

Using XGBoost on Amazon SageMaker

XGBoost is an [open source library available to download](#) and run almost anywhere, so it is also available in Amazon SageMaker. Amazon SageMaker is a managed training and hosting platform for machine learning workflows. Developers and data scientists use Amazon SageMaker to train and deploy machine learning models without the need to manage any infrastructure. Although you can always bring your own training and hosting containers to the Amazon SageMaker platform, you can also choose to leverage the algorithms and libraries that come with SageMaker, including XGBoost. There are many reasons why you might consider using XGBoost on Amazon SageMaker:

- **Out-of-the-box distributed training.** Amazon SageMaker with XGBoost allows customers to train massive data sets on multiple machines. Just specify the number and size of machines on which you want to scale out, and Amazon SageMaker will take care of distributing the data and training process.
- **Sharded by Amazon S3 key training.** Sharded by Amazon S3 key training requires you to partition your data on Amazon S3. This allows Amazon SageMaker to download each partition of the dataset to individual nodes rather than downloading all the data on all nodes. This saves time in downloading the dataset from Amazon S3 and ultimately speeds up training jobs.
- **XGBoost Instance Weighted Training.** Using XGBoost on SageMaker allows you to add weights to individual data points, also referred to as *instances*, while training. This allows customers to differentiate the importance of different instances during model training by assigning them weight values.
- **Spark integration with the Spark SDK.** The [SageMaker Spark SDK](#) provides a concise API for developers to interact with Amazon SageMaker XGBoost. Developers can first preprocess data on Apache Spark, then call Amazon SageMaker XGBoost directly from their Spark environment. This spins up the Amazon SageMaker training instances and uses them to train models on the data that was already preprocessed with Spark.
- **Easy deployment and managed model hosting.** After a model is trained, you need only one API call to deploy it to production. The Amazon SageMaker hosting environment is managed, and it [can be configured for auto scaling](#), which reduces the operational overhead of running a hosting environment.
- **Native A/B testing.** Using Amazon SageMaker hosting, you can run multiple XGBoost models, each with different weights for inference. The A/B testing helps customers determine the best models for their use case.

Tutorial – Predicting default on credit card payments

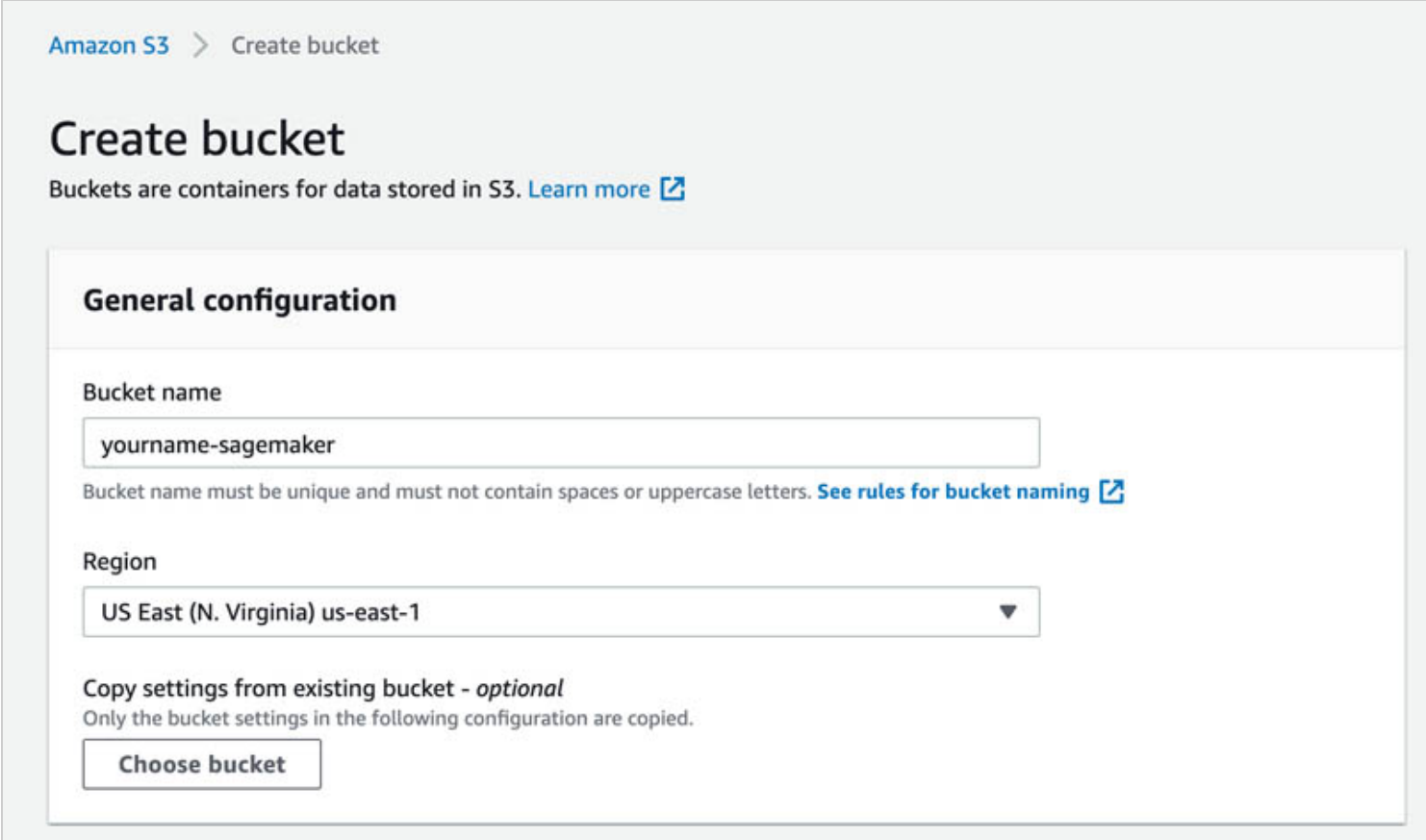
In this tutorial, we are going to leverage Amazon SageMaker to train and deploy an XGBoost model that is able to detect whether or not a person is going to default on their credit card payment. We use the “default of credit card clients” [dataset¹](#), from the UCI machine learning repository.

To start, we first create an Amazon S3 bucket. This stores our training dataset and the model that Amazon SageMaker outputs after the training job is complete. Next, we will create an Amazon SageMaker notebook instance. The notebook instance provides us with a managed Jupyter notebook environment to download the dataset, preprocess the data, train the model, host the model, and make predictions.

First, go to the Amazon S3 console at <https://s3.console.aws.amazon.com/>.

Choose the **Create bucket** button.

In the General configuration section, give your bucket a name, such as “yourname-sagemaker”.



Amazon S3 > Create bucket

Create bucket

Buckets are containers for data stored in S3. [Learn more](#)

General configuration

Bucket name

Bucket name must be unique and must not contain spaces or uppercase letters. [See rules for bucket naming](#)

Region

US East (N. Virginia) us-east-1 ▼

Copy settings from existing bucket - *optional*
Only the bucket settings in the following configuration are copied.

Choose bucket

Choose **Create bucket**.

Default encryption

Automatically encrypt new objects stored in this bucket. [Learn more](#)

Server-side encryption

☒ Disable
 ☐ Enable

Advanced settings

After creating the bucket you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel
Create bucket

Take a note of the Region in which you created the bucket, then navigate to the Amazon SageMaker console at <https://console.aws.amazon.com/sagemaker/> in the same Region.

Choose **Notebook instances**.

Amazon SageMaker

Amazon SageMaker Studio

Dashboard

Search

Images

Ground Truth

Labeling jobs

Labeling datasets

Labeling workforces

Notebook

Notebook instances

Lifecycle configurations

Git repositories

Processing

Processing jobs

Training

Amazon SageMaker > Dashboard

Overview

Ground Truth
Set up and manage labeling jobs for highly accurate training datasets using active learning and human labeling.

Labeling jobs

Notebook
Availability of AWS and SageMaker SDKs and sample notebooks to create training Jobs and deploy models.

Notebook instances

Training
Train and tune models at any scale. Leverage high performance AWS algorithms or bring your own.

Training jobs

Hyperparameter tuning jobs

Inference
Create models from training jobs or import external models for hosting to run inferences on new data.

Models

Endpoints

Batch transform jobs

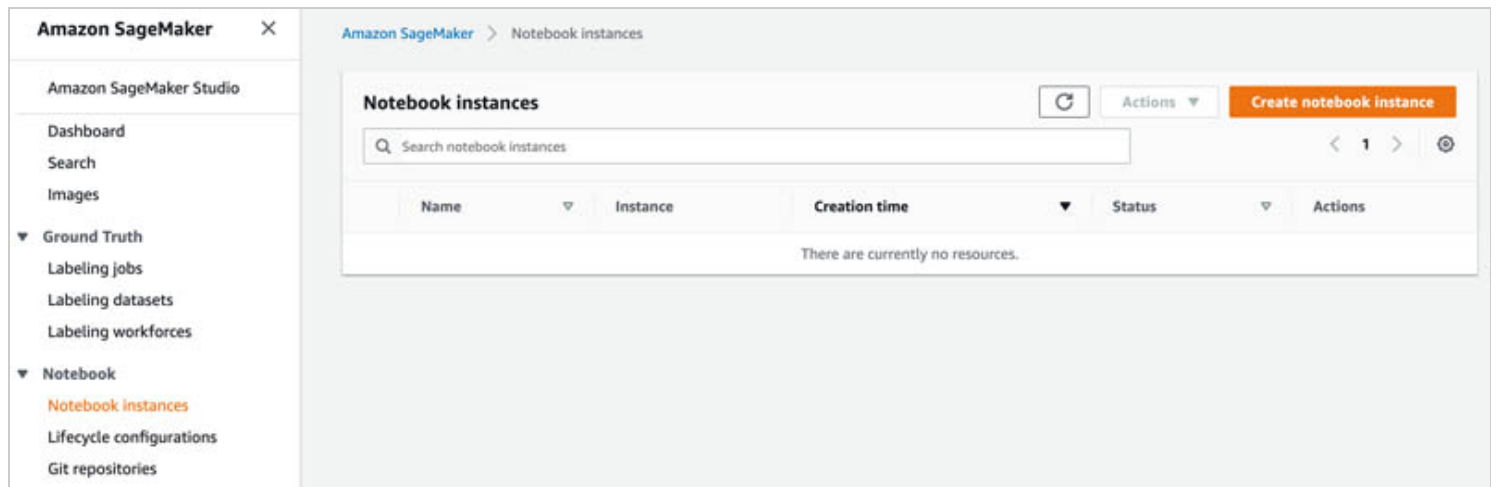
Processing Run
Pre- or post-processing and model evaluation workloads with a fully managed experience.

Processing jobs

Choose **Create notebook instance**.

<https://aws.amazon.com/blogs/machine-learning/simplify-machine-learning-with-xgboost-and-amazon-sagemaker/>

4/16



On the screen that appears, give your notebook a name, such as “yourname-notebook”.

In the IAM role drop-down list, choose **Create a new role**.

Notebook instance settings

Notebook instance name

Maximum of 63 alphanumeric characters. Can include hyphens (-), but not spaces. Must be unique within your account in an AWS Region.

Notebook instance type

ml.t2.medium ▼

Elastic Inference [Learn more](#) 

none ▼

► **Additional configuration**

Permissions and encryption

IAM role

Notebook instances require permissions to call other services including SageMaker and S3. Choose a role or let us create a role with the [AmazonSageMakerFullAccess](#) IAM policy attached.

Choose an IAM role ▼

Root access - optional

☒ **Enable** - Give users root access to the notebook

☐ **Disable** - Don't give users root access to the notebook

Create the new role by specifying the name of your bucket in the **Specific S3 bucket** text box. This gives Amazon SageMaker permission to access your S3 bucket.

Choose **Create role**.

Create an IAM role

Passing an IAM role gives Amazon SageMaker permission to perform actions in other AWS services on your behalf. Creating a role here will grant permissions described by the [AmazonSageMakerFullAccess](#) IAM policy to the role you create.

The IAM role you create will provide access to:

- ☒ **S3 buckets you specify - optional**
 - ☒ **Specific S3 buckets**

Comma delimited. ARNs, "*" and "/" are not supported.
 - ☐ **Any S3 bucket**
Allow users that have access to your notebook instance access to any bucket and its contents in your account.
 - ☐ **None**
- ☒ **Any S3 bucket with "sagemaker" in the name**
- ☒ **Any S3 object with "sagemaker" in the name**
- ☒ **Any S3 object with the tag "sagemaker" and value "true"** [See Object tagging](#)
- ☒ **S3 bucket with a Bucket Policy allowing access to SageMaker** [See S3 bucket policies](#)

CancelCreate role

Choose **Create notebook instance**.

Permissions and encryption

IAM role

Notebook instances require permissions to call other services including SageMaker and S3. Choose a role or let us create a role with the [AmazonSageMakerFullAccess](#) IAM policy attached.

AmazonSageMaker-ExecutionRole-20210102T191422 ▼



Success! You created an IAM role.
[AmazonSageMaker-ExecutionRole-20210102T191422](#) 



Root access - optional

☒ **Enable** - Give users root access to the notebook

☐ **Disable** - Don't give users root access to the notebook
Lifecycle configurations always have root access

Encryption key - optional

Encrypt your notebook data. Choose an existing KMS key or enter a key's ARN.

No Custom Encryption ▼

► **Network - optional**

► **Git repositories - optional**

► **Tags - optional**

Cancel

Create notebook instance

After the notebook is ready, you have the option to open the notebook.

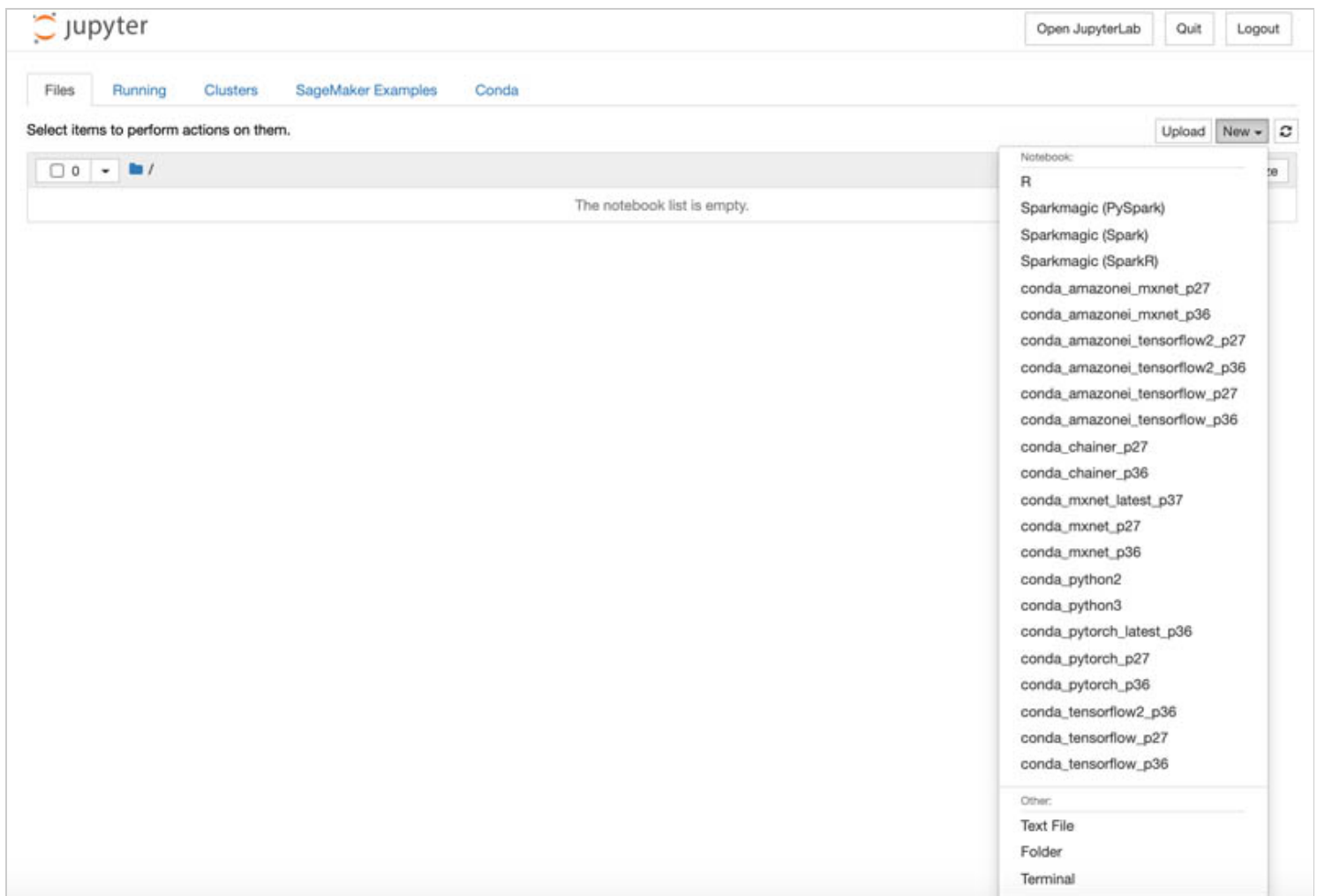
Choose **Open Jupyter**.

Notebook instances						 Actions	Create notebook instance
Search notebook instances						< 1 ... >	
Name	Instance	Creation time	Status	Actions			
☐ yourname-notebook	mlt2.medium	Jan 02, 2021 17:09 UTC	InService	Open Jupyter Open JupyterLab			

The SageMaker notebook instance comes with various sample notebooks, but in this tutorial, we'll be authoring a new notebook.

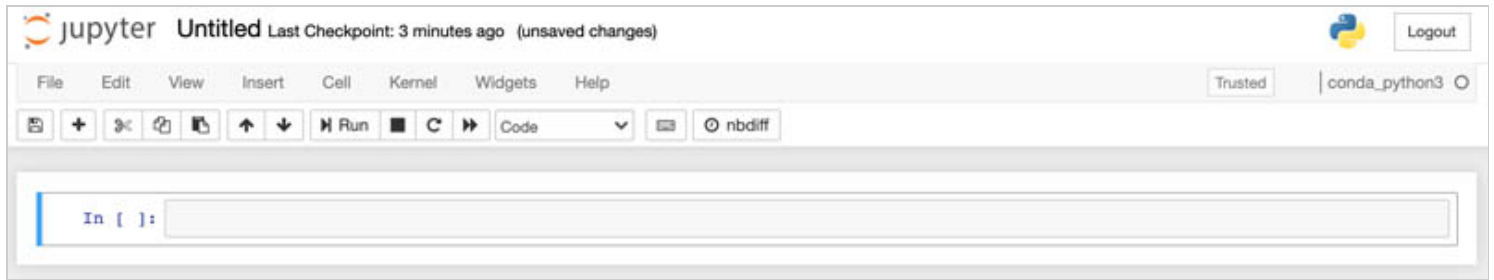
Choose the **New** button.

Choose **conda_python3** to open a new notebook with a Python3.6 environment.



The screenshot shows the JupyterLab interface. At the top, there are buttons for 'Open JupyterLab', 'Quit', and 'Logout'. Below these are tabs for 'Files', 'Running', 'Clusters', 'SageMaker Examples', and 'Conda'. The 'Files' tab is active, showing a file browser with a message 'The notebook list is empty.' A 'New' button is visible in the top right of the file browser. A dropdown menu is open from the 'New' button, listing various notebook environments. The list includes 'R', 'Sparkmagic (PySpark)', 'Sparkmagic (Spark)', 'Sparkmagic (SparkR)', and several 'conda' environments with different frameworks and versions (e.g., 'conda_amazonei_mxnet_p27', 'conda_amazonei_tensorflow2_p27', 'conda_amazonei_tensorflow2_p36', 'conda_amazonei_tensorflow_p27', 'conda_amazonei_tensorflow_p36', 'conda_chainer_p27', 'conda_chainer_p36', 'conda_mxnet_latest_p37', 'conda_mxnet_p27', 'conda_mxnet_p36', 'conda_python2', 'conda_python3', 'conda_pytorch_latest_p36', 'conda_pytorch_p27', 'conda_pytorch_p36', 'conda_tensorflow2_p36', 'conda_tensorflow_p27', 'conda_tensorflow_p36'). At the bottom of the dropdown, there is an 'Other:' section with options for 'Text File', 'Folder', and 'Terminal'.

Now that you have a notebook, you can enter code to download and view your dataset, train the model, and make predictions.



Use the Jupyter notebook to enter the following code and complete the tutorial. Press the play button in Jupyter or Shift+Enter to run code in the selected notebook cell.

```
bucket = 'yourname-sagemaker'
prefix = 'sagemaker/xgboost_credit_risk'

# Define IAM role
import boto3
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import sagemaker
from sagemaker import get_execution_role
from sagemaker.inputs import TrainingInput
from sagemaker.serializers import CSVSerializer

role = get_execution_role()
```

First, you need to set up your notebook by defining the S3 bucket you're using, importing the libraries you will need, and getting the Amazon SageMaker execution IAM role from the notebook environment that you will need to access training jobs.

```
!wget https://archive.ics.uci.edu/ml/machine-learning-databases/00350/default%20of%20c
```

The next step is to download the dataset, which comes as a .xls file.

```
dataset = pd.read_excel('default of credit card clients.xls')
pd.set_option('display.max_rows', 8)
```

```
pd.set_option('display.max_columns', 15)
dataset
```

Unnamed: 0		X1	X2		X3		X4	X5	X6	...		X18	X19	X20	X21	X22		X23		Y	
0	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	...	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6						default payment next month	
1	1	20000	2		2		1	24	2	...		0	689	0		0		0		1	
2	2	120000	2		2		2	26	-1	...		0	1000	1000		1000		0		2000	1
3	3	90000	2		2		2	34	0	...		1518	1500	1000		1000		1000		5000	0
...	...	...	...		...		...	...	...	...		...	...	...		...		...		...	...
29997	29997	150000	1		3		2	43	-1	...		1837	3526	8998		129		0		0	0
29998	29998	30000	1		2		2	37	4	...		0	0	22000		4200		2000		3100	1
29999	29999	80000	1		3		1	41	1	...		85900	3409	1178		1926		52964		1804	1
30000	30000	50000	1		2		1	46	0	...		2078	1800	1430		1000		1000		1000	1

30001 rows x 25 columns

By reading the dataset we see that it has 30,000 records, and each record has 23 associated attributes to describe features relevant to the credit scores of the person the record represents. The attributes are the following:

- X1: Amount of the given credit.
- X2: Gender (1 = male; 2 = female).
- X3: Education (1 = graduate school; 2 = university; 3 = high school; 4 = others).
- X4: Marital status (1 = married; 2 = single; 3 = others).
- X5: Age (year).
- X6 – X11: History of past payments. Tracked past monthly payment records (from April to September, 2005) are displayed as follows: X6 = the repayment status in September, 2005; X7 = the repayment status in August, 2005... X11 = the repayment status in April, 2005. The measurement scale for the repayment status is: -1 = pay duly; 1 = payment delay for one month; 2 = payment delay for two months... 8 = payment delay for eight months; 9 = payment delay for nine months and above.
- X12-X17: Amount of bill statement X12 = amount of bill statement in September, 2005; X13 = amount of bill statement in August, 2005... X17 = amount of bill statement in April, 2005.
- X18-X23: Amount of previous payment. X18 = amount paid in September, 2005; X19 = amount paid in August, 2005.... X23 = amount paid in April, 2005.
- Y: Did the person default? (Yes = 1, No = 0)

The “Y” attribute is known as the target attribute. This is the attribute that we want the XGBoost to predict. Because the target attribute is binary, our model will be performing binary prediction, also known as binary classification. In this dataset, a 1 in the Y column means that the person previously defaulted and 0 means that they have not defaulted in the past.

Normally, we would use feature engineering to select the best inputs for the model. *Feature engineering* is an iterative process that often requires experimentation and creating many models to find the input features that give the best model performance. For the purposes of this tutorial, we'll skip this step and train XGBoost on the features as they are given.

```
dataset = dataset.drop('Unnamed: 0', axis=1)
dataset = pd.concat([dataset['Y'], dataset.drop(['Y'], axis=1)], axis=1)
```

Amazon SageMaker XGBoost can train on data in either a CSV or LibSVM format. For this example, we use CSV. It should have the following:

- Have the target variable in the first column
- Not have a header row

To format our dataset for this, we drop the "ID" column that numbers the rows in the first column and we modify the "Y" column to be the first column in our DataFrame.

```
train_data, validation_data, test_data = np.split(dataset.sample(frac=1, random_state=), 3)
train_data.to_csv('train.csv', header=False, index=False)
validation_data.to_csv('validation.csv', header=False, index=False)
```

Here, we split our dataset into a training, validation, and testing set. XGBoost will train on the training dataset and use the validation set as data to evaluate prediction results as the model is trained. We will make predictions against the testing set after the model has been deployed.

```
boto3.Session().resource('s3').Bucket(bucket).Object(os.path.join(prefix, 'train/train
boto3.Session().resource('s3').Bucket(bucket).Object(os.path.join(prefix, 'validation/
s3_input_train = TrainingInput(s3_data='s3://{}/{}/train'.format(bucket, prefix), cont
s3_input_validation = TrainingInput(s3_data='s3://{}/{}/validation/'.format(bucket, pr
```

Now that our dataset is ready, we can upload it to Amazon S3 and take a note of its location so that we can use it for training.

```
containers = {'us-west-2': '433757028032.dkr.ecr.us-west-2.amazonaws.com/xgboost:lates
            'us-east-1': '811284229777.dkr.ecr.us-east-1.amazonaws.com/xgboost:lates
            'us-east-2': '825641698319.dkr.ecr.us-east-2.amazonaws.com/xgboost:lates
            'eu-west-1': '685385470294.dkr.ecr.eu-west-1.amazonaws.com/xgboost:lates
```

```
sess = sagemaker.Session()
```

```
xgb = sagemaker.estimator.Estimator(containers[boto3.Session().region_name],  
                                     role,  
                                     instance_count=1,  
                                     instance_type='ml.m4.xlarge',  
                                     output_path='s3://{}/{}/output'.format(bucket, pre  
sagemaker_session=sess)
```

```
xgb.set_hyperparameters(eta=0.1, objective='binary:logistic', num_round=25)  
xgb.fit({'train': s3_input_train, 'validation': s3_input_validation})
```

This step kicks off our XGBoost training job. We first define the location of the Amazon SageMaker XGBoost training containers. We then create an Amazon SageMaker estimator. By simply changing values such as `instance_count` and `instance_type`, we can change the size and number of instances we want to run on, which scales and distributes the training.

XGBoost also has a number of hyperparameters that we can tune to improve model performance. Here, we set values for some of the most commonly tuned hyperparameters. Notice the `objective` parameter is set to `binary:logistic`. This parameter tells XGBoost what kind of problem we are solving (classification, regression, ranking, etc.). In this case we are solving a binary classification problem –predicting whether or not a person is likely to default on their credit card payments.

When the training is complete, we'll see a log of the training. This log contains metrics, such as our training and validation errors and helps us gauge the performance of our model. The training logs are also available in Amazon CloudWatch Logs.

```
xgb_predictor = xgb.deploy(  
    initial_instance_count = 1,  
    instance_type = 'ml.m4.xlarge',  
    serializer = CSVSerializer())
```

After the model is trained, this deploys our model to Amazon SageMaker hosting. It takes a few minutes to set up the hosting endpoint.

```
def predict(data, rows=500):  
    split_array = np.array_split(data, int(data.shape[0] / float(rows) + 1))  
    predictions = ''  
    for array in split_array:  
        predictions = ','.join([predictions, xgb_predictor.predict(array).decode('utf-8')])  
  
    return np.fromstring(predictions[1:], sep=',')  
  
predictions = predict(test_data.to_numpy()[0:1000])  
predictions
```

Our final step is to make predictions. We first set up our serializer. This takes numpy arrays (like our testing data) and serializes them to CSV format. We then define a predict function that will take in our testing data as input and run predictions on the data, which is done through the `xgb_predictor.predict()` function call.

```
Out[97]: array([ 0.59283876,  0.1414548 ,  0.31579986, ...,  0.41416201,  
                0.11323805,  0.11232713])
```

The output is an array with our predictions. The first element of the array is the probability that the person corresponding to the first inputted data row will default on their credit card payments. The array continues with probabilities for all rows in our testing set.

Now that the model is hosted we can start to make predictions about which credit card holders will default on our payments. This allows us to understand the creditworthiness of our customers which can help us better issue credit and predict future payments.

Clean up

To avoid unnecessary charges on your AWS account, delete SageMaker endpoint you created earlier.

```
xgb_predictor.delete_endpoint()
```

Conclusion

XGBoost is a powerful machine learning library that is great for solving classification, regression, and ranking problems. Using XGBoost on Amazon SageMaker provides additional benefits like distributed training and managed model hosting without having to set up and manage any infrastructure. If you have a use case that XGBoost can solve, take a look at the sample notebooks:

- [Targeted Direct Marketing](#)– predicts potential customers that are most likely to convert based on customer and aggregate level metrics, using Amazon SageMaker’s implementation of
- [Predicting Customer Churn](#)– uses customer interaction and service usage data to find those most likely to churn, and then walks through the cost/benefit trade-offs of providing retention incentives. This uses Amazon SageMaker’s implementation of XGBoost to create a highly predictive model.

These notebooks contain examples on how to implement XGBoost, including examples of how the algorithm can be adapted for other use cases.

References

1. Yeh, I. C., & Lien, C. H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. Expert Systems with Applications, 36(2), 2473-2480.

About the Authors



Eitan Sela is a Solutions Architect with Amazon Web Services. He works with AWS customers to provide guidance and technical assistance, helping them improve the value of their solutions when using AWS. Eitan also helps customers build and operate machine learning solutions on AWS. In his spare time, Eitan enjoys jogging and reading the latest machine learning articles.



Yash Pant is an Enterprise Solutions Architect with AWS. He helps enterprise customers migrate and architect applications for the cloud, often focusing on machine learning workloads.



Saksham Saini is a Software Developer with Amazon Sagemaker. He did his BS in Computer Engineering from UIUC. He is currently working on building highly optimized and scalable algorithms for Amazon Sagemaker. Outside work, he enjoys reading, playing music and traveling.



Yijie Zhuang is a Software Engineer with AWS SageMaker. He did his MS in Computer Engineering from Duke. He is currently working on building algorithms and applications for Amazon SageMaker.



Comments

Log in to comment